



CIN0136

Desenvolvimento de Software

Desenvolvimento de Software

Prof. Vinicius Cardoso Garcia

vcg@cin.ufpe.br :: @vinicius3w :: assertlab.com

<http://viniciusgarcia.me>



**Centro de
Informática**
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO

Licença do material

Este Trabalho foi licenciado com uma Licença



**Atribuição-NãoComercial-
Compartilha Igual 4.0 Internacional
(CC BY-NC-SA 4.0)**

Mais informações visite

<https://creativecommons.org/licenses/by-nc-sa/4.0/deed.pt> BR



Engenharia de Software em Dimensões

É um material abrangente, didático e atualizado que apresenta a Engenharia de Software como uma disciplina multidimensional. Cada capítulo foca em uma dimensão prática, conectando teoria e realidade profissional com clareza, profundidade e propósito.

Prof. Vinicius Cardoso Garcia

<https://bit.ly/esdbook>

Engenharia de Software em Dimensões

Da base conceitual à prática profissional



Vinicius Cardoso Garcia



Ponto de Commit

O podcast figital da disciplina (IF977) Engenharia de Software do curso de bacharelado em Sistemas de Informação do Centro de Informática (CIn) da Universidade Federal de Pernambuco (UFPE).

<https://go.uaite.xyz/ponto-de-commit>

Ponto de Commit



O podcast figital da disciplina de Engenharia de Software



Engenharia de Software Moderna

Engenharia de Software Moderna é um livro destinado a alunos de cursos de graduação em Computação. Ele pode ser lido também por profissionais que buscam um conhecimento horizontal da área.

Prof. Marco Tulio Valente

<https://engsoftmoderna.info>

@engsoftmoderna

ENGENHARIA — DE — SOFTWARE MODERNA

Princípios e práticas para
desenvolvimento de software
com produtividade



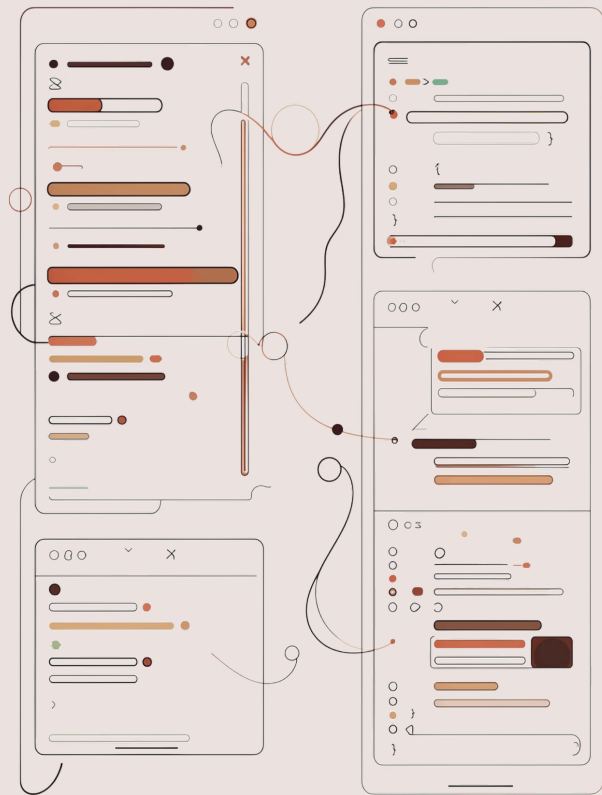
MARCO TULIO VALENTE





Refactoring: A Arte de Melhorar o Código Sem Quebrar Tudo

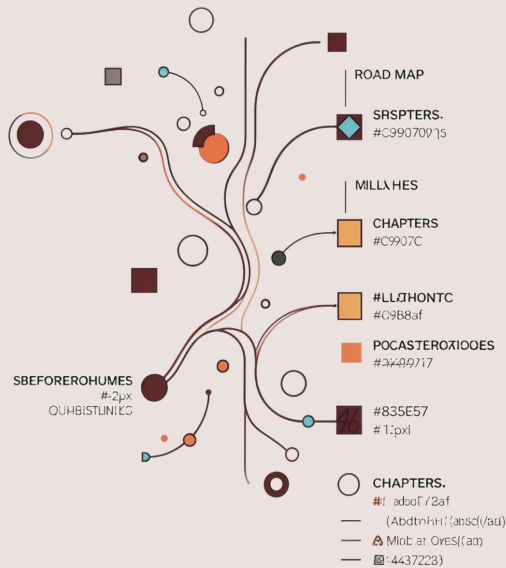
Compreenda o que é refactoring, como realizá-lo com segurança e por que testes são seu maior aliado nessa jornada.



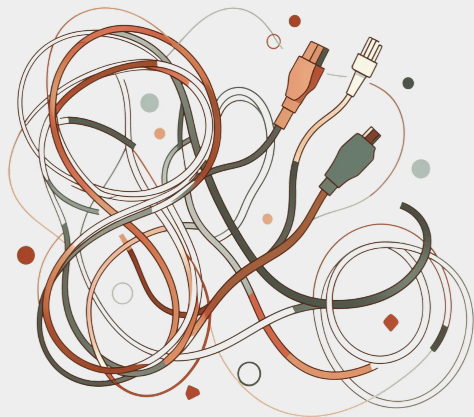


O Que é Refactoring e Por Que Ele é Essencial?

Antes de escrever qualquer código novo, vale a pena entender como cuidar bem do código que já existe.



O Código Que Você Escreveu Ontem...



→ Já é código legado

O tempo passa, os requisitos mudam e o código envelhece mais rápido do que imaginamos.

→ Manutenção domina o ciclo de vida

Cerca de **80% do ciclo de vida** de um software é gasto em manutenção, não em criação.

→ "Funciona" não é suficiente

Código que funciona pode ser um pesadelo para manter, estender ou depurar.



Refactoring: A Mudança Estrutural Segura

Definição

Processo de reestruturar código existente **sem alterar seu comportamento externo**, o resultado final permanece idêntico para o usuário.

Objetivo

Tornar o código mais **legível**, **compreensível**, **manutenível** e **extensível**, reduzindo a carga cognitiva da equipe.

O que NÃO é

Não é adicionar novas funcionalidades, nem corrigir bugs, embora um código mais limpo **facilite muito ambos**.



Por Que Refatorar? Os Benefícios Tangíveis



Redução de Bugs

Código limpo tem menos "cantos escuros" onde bugs se escondem. A lógica clara expõe erros antes que eles causem problemas.



Aumento de Produtividade

Desenvolvedores entendem e modificam o código mais rapidamente, gastando menos tempo decifrando intenções obscuras.



Onboarding Mais Rápido

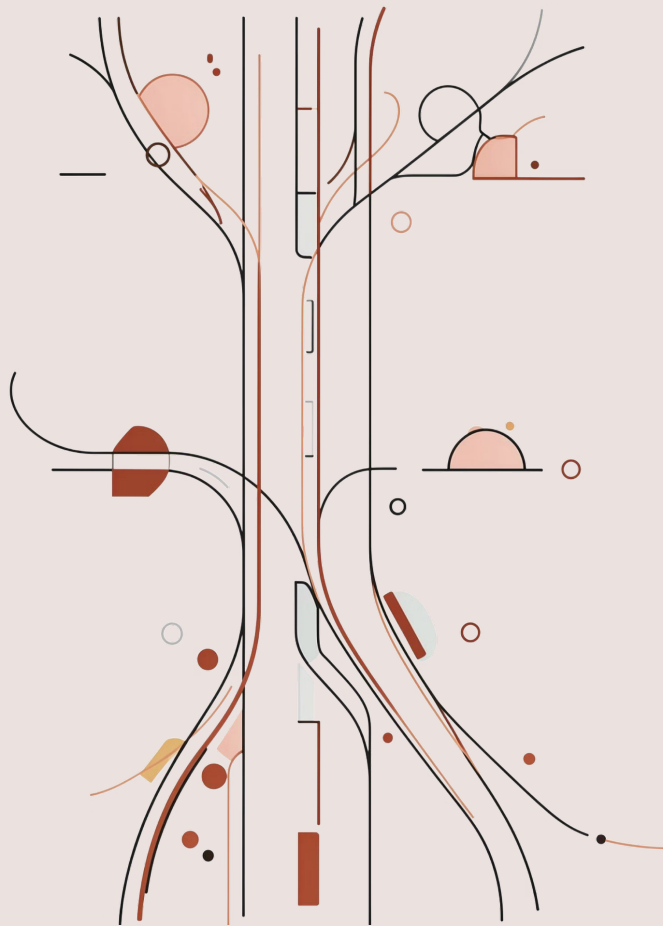
Novos membros da equipe se adaptam com mais facilidade quando o código comunica claramente sua intenção.



Menor Estresse

Lançamentos e manutenções se tornam menos assustadores quando a base de código é confiável e compreensível.





Do Caos à Clareza

Refactoring é a jornada de transformar código confuso em algo que qualquer desenvolvedor entende de relance.





A Regra de Ouro: Testes São Seu Para-quedas

Nenhuma refatoração é segura sem uma rede de proteção. Testes automatizados são essa rede.



Refactoring Sem Testes: Andando no Escuro

A ausência de testes transforma cada mudança em um salto de fé. O risco de introduzir regressões é altíssimo e a confiança para evoluir o código desaparece gradualmente.

"Refatorar sem testes é como tentar consertar um carro em alta velocidade sem freios."

"Funciona... por enquanto."

A falsa sensação de segurança é o maior inimigo do código sem testes.

Regressões silenciosas

Comportamentos que antes funcionavam podem quebrar de forma sutil e imperceptível.

Medo paralisa a evolução

Sem testes, o time prefere não tocar no código e a dívida técnica cresce.



A Importância dos Testes em JavaScript

JavaScript é uma linguagem dinâmica com inúmeras peculiaridades. A detecção estática de erros é limitada, tornando os testes automatizados ainda mais críticos.

Tipagem dinâmica

Erros de tipo só aparecem em tempo de execução, testes capturam isso antes do deploy.

Comportamentos implícitos

Coerção de tipos, closures e escopo podem gerar surpresas que só testes revelam.

Cobertura como proteção

Cobertura de testes adequada é pré-requisito para refatorar JavaScript com segurança.



Testes de Caracterização

Escritos **antes de qualquer mudança**, esses testes documentam o comportamento existente, mesmo que seja incorreto. Eles são um espelho fiel do que o código faz hoje.

❗ Exemplo: testar `processOrder()` com entradas específicas para capturar e fixar o resultado atual como expectativa.

1

Execute o código atual

Observe a saída produzida para um conjunto representativo de entradas.

2

Fixe a expectativa

Defina a saída observada como o resultado esperado no teste, sem julgamentos.

3

Refatore com segurança

Qualquer mudança que altere o comportamento fará o teste falhar, um sinal de alerta imediato.



Testes: Sua Rede de Segurança

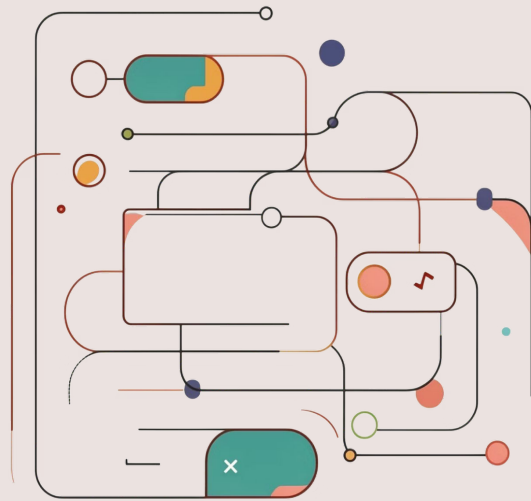
Com testes, você refatora com confiança. Sem eles, cada mudança é uma aposta.





Ferramentas e Técnicas de Refactoring em JavaScript

Da renomeação de variáveis à eliminação de duplicação, conheça as técnicas essenciais com exemplos práticos em JS.



Renomear Variáveis e Funções

O primeiro e mais impactante passo. Nomes claros revelam a intenção do código sem necessidade de comentários.

- ✓ IDEs modernas como **VS Code** e **JetBrains** oferecem renomeação segura que atualiza todas as referências automaticamente.

Antes

```
function doThing(a, b) {  
    return a + (a * b);  
}
```

Depois

```
function calculateTotal(price, tax) {  
    return price + (price * tax);  
}
```

Baixo risco, alto impacto — a mudança mais simples que transforma a legibilidade do código.



Extrair Função: Dividir para Conquistar

Mover um bloco de código repetido ou complexo para uma nova função reduz duplicação, melhora a legibilidade e facilita testes unitários isolados.

Testabilidade

Funções pequenas e focadas são muito mais fáceis de testar isoladamente.

Menos duplicação

Uma mudança num único lugar reflete em todo o sistema.

Antes

```
function processOrder(order) {  
  let total = 0;  
  for (let item of order.items) {  
    total += item.price * item.qty;  
  }  
  total += total * 0.1; // tax  
  return total;  
}
```

Depois

```
function calculateOrderTotal(items) {  
  const subtotal = items.reduce(  
    (sum, item) => sum + item.price * item.qty, 0  
  );  
  return applyTax(subtotal, 0.1);  
}  
  
function applyTax(value, rate) {  
  return value + (value * rate);  
}
```



Simplificar Condicionais: Clareza na Lógica

1

Funções booleanas nomeadas

Substitua condições complexas por funções com nomes que explicam a intenção:

```
isEligibleForDiscount().
```

2

Early Returns

Retorne cedo para evitar aninhamentos excessivos de `if/else`, o "código pirâmide" que dificulta a leitura.

3

Extrair lógica

Mova a lógica de condicionais complexos para funções separadas com nomes descritivos e bem testáveis.

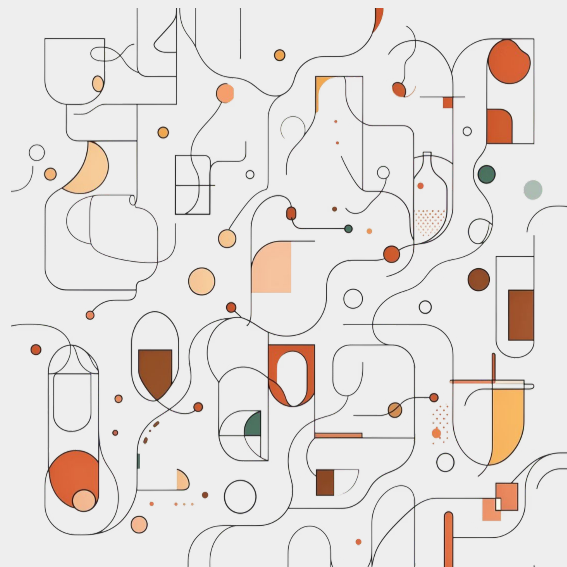
- ❏ Regra prática: se você precisa de um comentário para explicar um `if`, é hora de extraí-lo para uma função nomeada.



Eliminar Duplicação: O Princípio DRY

Don't Repeat Yourself — toda duplicação é uma bomba-relógio.

Quando a lógica mudar, você precisará lembrar de todos os lugares onde a copiou.



Extraíir funções

A técnica mais direta: mova o código repetido para uma função reutilizável.

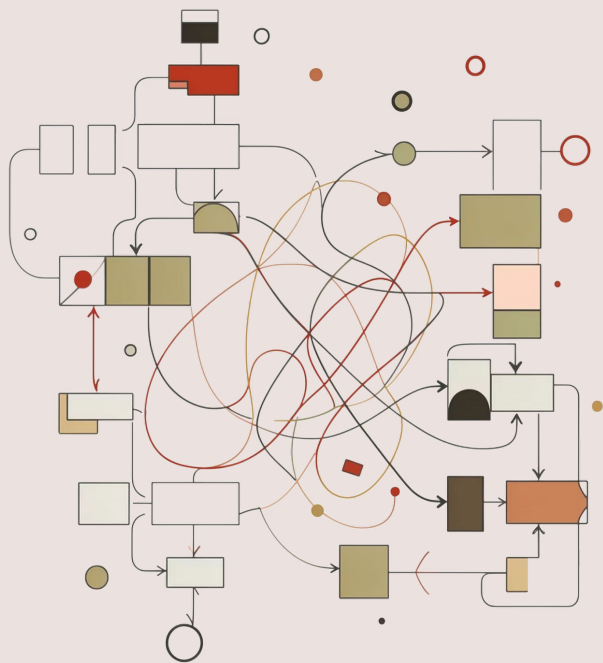
Encapsular em classes

Use objetos para agrupar lógica comum e compartilhá-la de forma coesa.

Padrões de design

Factory, Strategy e Template Method são aliados poderosos para eliminar redundâncias estruturais.





Simplificando a Complexidade

Cada técnica de refactoring é um passo na direção de um código que qualquer desenvolvedor consegue seguir, sem mapa e sem guia.





Identificando "Code Smells": Sinais de Alerta

Aprenda a reconhecer os sintomas que indicam que uma área do código precisa de atenção antes que vire um problema maior



O Que São "Code Smells"?

Code smells são **indicadores de que algo pode estar errado** com o código. Não são bugs, o código pode funcionar perfeitamente, mas são sintomas que sugerem a necessidade de refactoring.

"Um cheiro no código é como uma dor leve: ignorá-lo hoje garante uma cirurgia amanhã."

Não são bugs

O código executa, mas a estrutura dificulta a manutenção e a evolução futura.

São sintomas

Indicam problemas de design subjacentes que tendem a piorar com o tempo.

Guiam o refactoring

Um catálogo de smells ajuda a priorizar onde focar seus esforços de melhoria.



Code Smells Comuns (e Como Lidar com Eles)

1

Duplicated Code

Eliminar com extração de funções ou classes que centralizam a lógica repetida.

2

Long Method

Dividir em funções menores e focadas, cada função deve fazer uma coisa só.

3

Large Class

Dividir em classes menores com responsabilidades únicas (Princípio SRP).

4

Feature Envy

Função mais interessada nos dados de outra classe.
Solução: mová-la para lá.

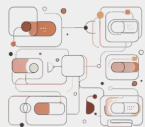
5

Primitive Obsession

Usar `string` onde um objeto `Email` seria mais semântico e seguro.



Mais Code Smells para Ficar de Olho



Switch Statements

Frequentemente indicam a necessidade de polimorfismo, substitua por classes e interfaces.



Data Clumps

nome, sobrenome, email sempre juntos? Encapsule-os em um objeto `Contact`.



Lazy Class

Classes que fazem pouco demais. Funda-as com outra classe para eliminar a indireção desnecessária.



Excessive Comments

Comentários em excesso compensam código obscuro. Refatore para que o código fale por si mesmo.



Sinais de Alerta: Ouça o Código!

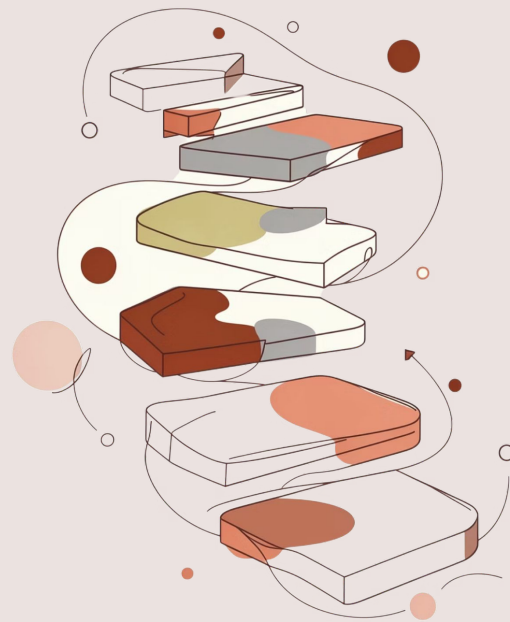
Code smells não gritam, eles sussurram.
Desenvolvedores experientes aprendem a
ouvir esses sussurros antes que virem
problemas reais.





Refactoring Incremental e o Padrão Strangler Fig

Grandes transformações começam com pequenos passos seguros. Conheça as estratégias para refatorar com confiança em qualquer escala.

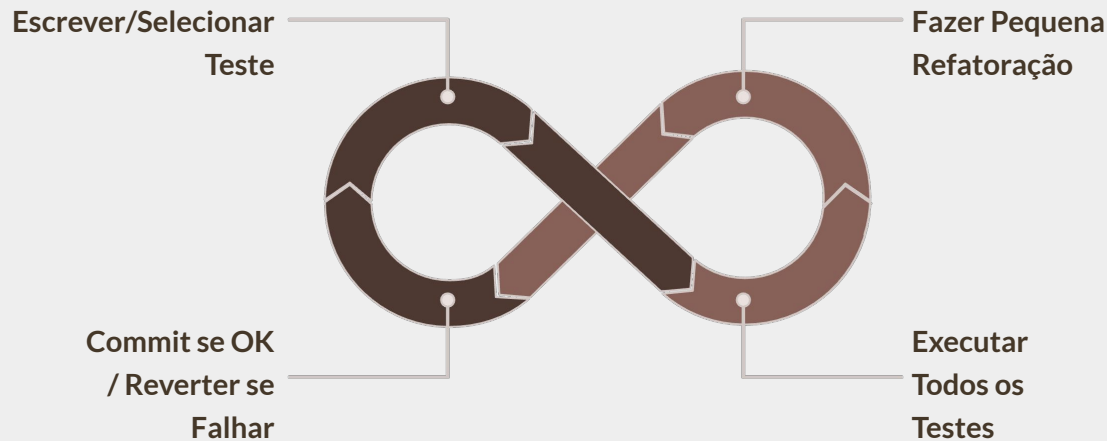


O Método do Pequeno Passo

O princípio central é simples: **faça uma pequena mudança de cada vez.**

Cada passo deve ser verificável, reversível e comitável de forma independente.

Isso minimiza o risco, facilita revisões de código com PRs menores e cria um histórico de mudanças rastreável e compreensível.



O Padrão Strangler Fig

Inspirado na figueira estranguladora que cresce ao redor de uma árvore antiga, este padrão permite substituir sistemas legados **sem interromper a operação**.

1

Identifique

Escolha uma funcionalidade específica a ser refatorada — comece pelas menos críticas.

2

Implemente em paralelo

Construa a nova versão ao lado da antiga, sem tocar no código legado.

3

Redirecione o tráfego

Migre gradualmente os usuários para a nova implementação, monitorando de perto.

4

Remova o legado

Quando a nova versão estiver comprovada, elimine o código antigo com segurança.

 Ideal para grandes refatorações em sistemas legados críticos onde não é possível parar para reescrever tudo de uma vez.





Substituição Gradual e Segura

O Strangler Fig garante que o sistema nunca pare, enquanto o novo cresce, o velho encolhe até desaparecer.





Refactoring em JavaScript Legado (Sem Testes)

A realidade de muitos projetos: código funcional, sem cobertura de testes. Não é ideal, mas existem estratégias para avançar com segurança.



Estratégias para Código Sem Testes

1

Logging estratégico

Adicione logs em pontos-chave para observar o fluxo de dados e valores antes de qualquer mudança.

2

Reprodução manual

Documente o comportamento atual testando manualmente os casos de uso mais importantes.

3

Testes mínimos primeiro

Adicione testes unitários focados nas partes que serão tocadas, antes de refatorar qualquer linha.

4

Ferramentas de análise

Use linters (ESLint) e ferramentas de análise estática para identificar problemas sem executar o código.

⚠ Nunca refatore código sem testes em produção sob pressão. Prefira criar os testes primeiro, mesmo que demande mais tempo inicial.





Navegando no Desconhecido com Cautela

Sem testes, cada movimento precisa ser deliberado, documentado e reversível. A cautela não é fraqueza, é profissionalismo.



Conclusão: Refactoring é um Investimento Contínuo

Prática contínua

Refatorar não é um evento único. É um hábito que separa equipes de alta performance das demais.

Confiança com testes

A segurança para refatorar cresce com a cobertura de testes. Um complementa o outro.

Código que você ama

Código limpo é código que sua equipe terá prazer em trabalhar, hoje, amanhã e daqui a dois anos.

- ✅ **Chamada para Ação:** Comece pequeno. Escreva um teste. Faça uma pequena melhoria. Abrace a melhoria contínua como parte da sua identidade como desenvolvedor.



contacts

vcg@cin.ufpe.br
[@vinicius3w](https://www.instagram.com/vinicius3w)
viniciusgarcia.me

academy

go.uaite.xyz/vcg-es
go.uaite.xyz/vcg-microservices
go.uaite.xyz/vcg-devops
go.uaite.xyz/vcg-esaia
go.uaite.xyz/vcg-deso

Desenvolvimento de Software (CIN0136)

Refactoring: A Arte de Melhorar o
Código Sem Quebrar Tudo